

ETX Plan v2: Beat vLLM on MoE Decode

Author: Kailun Chen · Status: v2 plan, supersedes PLAN-fusion-win.md · Date: 2026-09-25 · Evidence: design document sections 15.5-15.7, examples/llm/results/

0. Summary

- **Change of target.** Stop trying to prove fusion on batch-1 dense decode. Measurement showed it is a barrier chain with nothing to overlap; a megakernel can at best equal a HIP graph there.
- **New target.** Qwen3-30B-A3B (128 experts, top-8) decode on one MI300X: **at most 3.5 ms per token** at batch 1, against vLLM’s best 4.84 ms; and ahead of vLLM at batch 4-8. Every token identical to HF.
- **Why MoE.** vLLM reaches only 26% of bandwidth on this model (6.1 GB read per token in 4.84 ms; the floor is about 1.5 ms) against 60% on the dense 8B. The loss comes from choosing experts only after the router, from small fragmented expert matrices, and from kernels that depend on the routing and so cannot be frozen into a graph. Those three are exactly what a megakernel is good at.
- **Two independent problems, solved separately.** Fusion gains are sought only in the MoE structure; tile speed is its own job, with targets of 3 TB/s for GEMV and under 5 us per layer for attention.
- **Budget.** Four phases, about six GPU sessions on a 1x MI300X at \$2.99/h, \$25-35 in total. Balance \$22.37.

1. Established facts

Fact	Evidence
With the same tiles, fused and HIP-graph unfused are equal on dense models (2-5% apart)	sliced graph, three models
The 2x gap to vLLM is entirely in the tiles: GEMV at 1.5-2.5 TB/s, attention plus merge 50 us per layer	Qwen3-8B per-phase trace
With good tiles the compiled kernel beats vLLM by 8% and is within 6% of hand-written	DeepSeek with fleet’s tiles
vLLM uses 26% of bandwidth on the MoE model	4.84 ms for 6.1 GB
One workgroup per CU is right for the persistent kernel; L2-warming prefetch loses; the relay’s hierarchical acquire is unsafe	measured 2026-09-24/25

2. What one MoE layer looks like

Today, sliced graph, one workgroup per CU: 219 us per layer on Qwen3-30B-A3B.

Phase	Today	Target	Means
qkv	22 us	10	GEMV tile
post (q/k norm, RoPE, KV write)	15	0	folded into qkv’s epilogue
attention	45	10	flash-decode layout
merge	22	0	done by each head’s last attention chunk
o_proj	21	5	small-K GEMV tile

Phase	Today	Target	Means
fold + router	15	8	already XCD-local
expert gate_up	51	18	expert GEMV tile
expert down	34	12	expert GEMV tile
two device-wide handoffs	6	6	-
total	219	about 70	48 layers about 3.4 ms, plus lm_head 0.2 ms

The structural fusion gain on MoE is already in the sliced graph: every XCD computes the router itself, there is no device-wide barrier between the router and the experts, and XCD k starts streaming its expert's weights the moment it knows them. vLLM runs router, top-k and fused_moe as separate kernels with boundaries between them, and fused_moe's grouped GEMM is built for large batches and inefficient for one token.

3. Phases

Phase 1: tile speed (2-3 sessions)

- **GEMV:** with one workgroup per CU the register budget is 512; raise loads in flight to 32-48 per lane; weight blocks of at least 128 KB per task, small matrices (o_proj, router) merged into fewer tasks; the input vector kept in LDS as bf16, which is exact because HF's linear inputs are bf16 already. Target 3 TB/s.
- **Attention:** four waves over position ranges, vectorised 128-dimension dot products, partial sums out; the last chunk of each head merges in place, removing the merge phase.
- **post folded into qkv:** qkv tasks cover whole heads and finish norm, RoPE and the KV write in their epilogue.
- Done when: fused Qwen3-30B-A3B at most 5 ms (vLLM parity), Qwen3-8B at most 6 ms; tokens identical to HF.

Phase 2: the MoE structural gain (1-2 sessions)

- **Expert weights prefetched into LDS:** once the XCD's fold task has the routing, the expert gate_up tasks pull their slice of the weights into an LDS staging area (about 30 KB free per CU) while waiting on the local event; no L2 warming.
- **Assignment when top-k differs from the XCD count:** experts distributed by load, not one slot per XCD.
- Done when: Qwen3-30B-A3B at most 3.5 ms, at least 25% ahead of vLLM.

Phase 3: small batches (1-2 sessions)

- Batch 4-8: tokens of one step grouped by expert, expressed with ETX's data-dependent edges ("b->topk[b, :]", indptr range maps), so each expert's weights are read once.
- Waiting at a handoff is filled by other tokens' tasks, which a graph cannot do.
- Done when: per-token time at batch 4 and 8 at most 80% of vLLM's at the same batch.

Phase 4: comparison and delivery (1 session)

- Qwen3-30B-A3B x {vLLM best, ETX fused, ETX unfused} x batch {1, 4, 8}, 1024-token context, three rounds each, every run checked token by token against HF.
- Update the design document, the brief and the README.

4. Risks

- **3 TB/s for expert GEMV is an estimate.** One expert is only 3 MB, 80 KB per task across 38 workers, so latency is harder to hide; at 2 TB/s the result is about 4.3 ms, still 11% ahead of vLLM.
- **LDS prefetch needs room.** The attention and GEMV input vectors already take most of the LDS; the depth is decided by A/B.
- **Small-batch routing groups touch the compiler's instantiation path.** That code was validated on the MoE example, not yet on a real model.
- **GPU availability.** Several-hour waits for a free VM in recent days.

5. Order

Phase 1 first. It answers two things at once: whether the tiles reach 3 TB/s, and whether the fused MoE kernel then already equals vLLM. If phase 1 clears its bar, phases 2 and 3 decide by how much ETX wins; if it does not, the problem is unambiguously tile engineering, and linking external tiles becomes the option.